

Business Model Canvas

Tema: Sistema de Gestão de Condomínio (Plataforma SaaS & Serviços)

Bloco 1 Segmentos de Clientes

Para quem estamos criando valor? Quem são nossos clientes-alvo?

- **Síndicos Profissionais e Orgânicos (Moradores):** Gestores que precisam otimizar o dia a dia do prédio com rapidez e transparência.
- **Administradoras de Condomínios:** Empresas que gerenciam dezenas de condomínios simultaneamente e precisam de automação em massa.
- **Moradores e Condôminos (Usuários finais):** Precisam de facilidades como reserva de áreas comuns, emissão de boletos e comunicação com a gestão.
- **Equipe Operacional (Portaria e Manutenção):** Funcionários focados no controle rigoroso de acessos e registro digital de encomendas.

Bloco 2 Proposta de Valor

O que entregamos de valor para o cliente? Qual dor resolvemos?

- **Para Síndicos e Administradoras:** Redução drástica da inadimplência por automação de cobranças, visibilidade financeira em tempo real e centralização operacional.
- **Para Moradores:** Praticidade total via app móvel para reservas, acompanhamento de contas e participação em assembleias virtuais.
- **Para a Portaria:** Maior segurança com controle de visitantes e registro digital de correspondências integradas.

Bloco 3 Canais

Como alcançamos nossos clientes? Como entregamos a solução?

- **Marketing Digital (Inbound):** Produção de conteúdo focado em gestão, SEO e campanhas voltadas para administradoras.
- **Equipe de Vendas (Inside Sales):** Demonstrações práticas direcionadas a tomadores de decisão e grandes construtoras.
- **Plataforma Própria (SaaS):** Acesso via aplicativo mobile (iOS/Android) e portal web dedicado.

Bloco 4 Relacionamento com Clientes

Como interagimos com cada segmento de cliente ao longo de sua jornada?

- **Suporte Técnico Humanizado & Automatizado:** Canais eficientes (WhatsApp, chat, central de ajuda e tutoriais em vídeo).
- **Onboarding Assistido:** Treinamento inicial estruturado para garantir que a equipe da administra ou novo síndico domine a plataforma rapidamente.
- **Autoatendimento:** Autonomia completa para o condômino emitir segunda via de boletos, consultar murais e efetuar reservas sem intervenção humana.

Bloco 5 Fontes de Receita

De que forma geramos receita a partir da proposta de valor?

- **Modelo de Assinatura (SaaS):** Mensalidade recorrente cobrada das administradoras ou condomínios com base na quantidade de unidades geridas.
- **Tarifas de Transação Financeira:** Percentual ou taxa fixa sobre o volume de boletos emitidos/compensados e meios de pagamento integrados.
- **Módulos Adicionais (Upsell):** Comercialização opcional de recursos avançados, como assembleias virtuais com validade jurídica ou controle biométrico/facial.

Bloco 6 Recursos-Chave

Quais ativos físicos, intelectuais e humanos nossa proposta exige?

- **Infraestrutura de TI e Nuvem:** Servidores robustos e seguros (AWS/Azure/GCP) que garantem alta disponibilidade e escalabilidade.
- **Propriedade Intelectual:** Código-fonte proprietário do software, arquitetura de dados e protocolos de segurança digital.
- **Equipe Especializada:** Desenvolvedores de software, especialistas em suporte, equipe comercial e consultores jurídicos imobiliários.

Bloco 7 Atividades-Chave

O que precisamos executar excelentemente para o modelo funcionar?

- **Desenvolvimento e Atualização Contínua:** Lançamento de melhorias de interface (UX), novas features e correções de bugs.
- **Segurança e Conformidade (LGPD):** Proteção rigorosa de dados pessoais e financeiros de condôminos e condomínios.
- **Suporte Contínuo e Vendas:** Resolução rápida de chamados operacionais e prospecção ativa de novos clientes.

Bloco 8 Parcerias-Chave

Quem são nossos principais parceiros e fornecedores estratégicos?

- **Gateways de Pagamento e Bancos:** Parceiros para emissão automatizada e processamento seguro de boletos e Pix.
- **Fabricantes de Hardware de Segurança:** Marcas de catracas, câmeras e sistemas de controle de acesso integrados via API.
- **Administradoras de Referência:** Empresas parceiras que validam a ferramenta no mercado e servem de cases de sucesso.

Bloco 9 Estrutura de Custos

Quais são os custos mais relevantes inerentes ao modelo de negócio?

- **Custos de Infraestrutura e Licenciamento:** Servidores em nuvem, ferramentas de desenvolvimento e APIs de terceiros.
- **Folha de Pagamento:** Remuneração da equipe técnica (TI), atendimento, suporte e time comercial.
- **Aquisição de Clientes (CAC) e Operações:** Investimentos em marketing digital, tráfego pago, contabilidade e assessoria jurídica.



Sistema de Gestão Condominial — Documentação de UX/UI & Wireframes



Protótipo e Wireframes no Figma



Acesse o projeto completo no Figma:

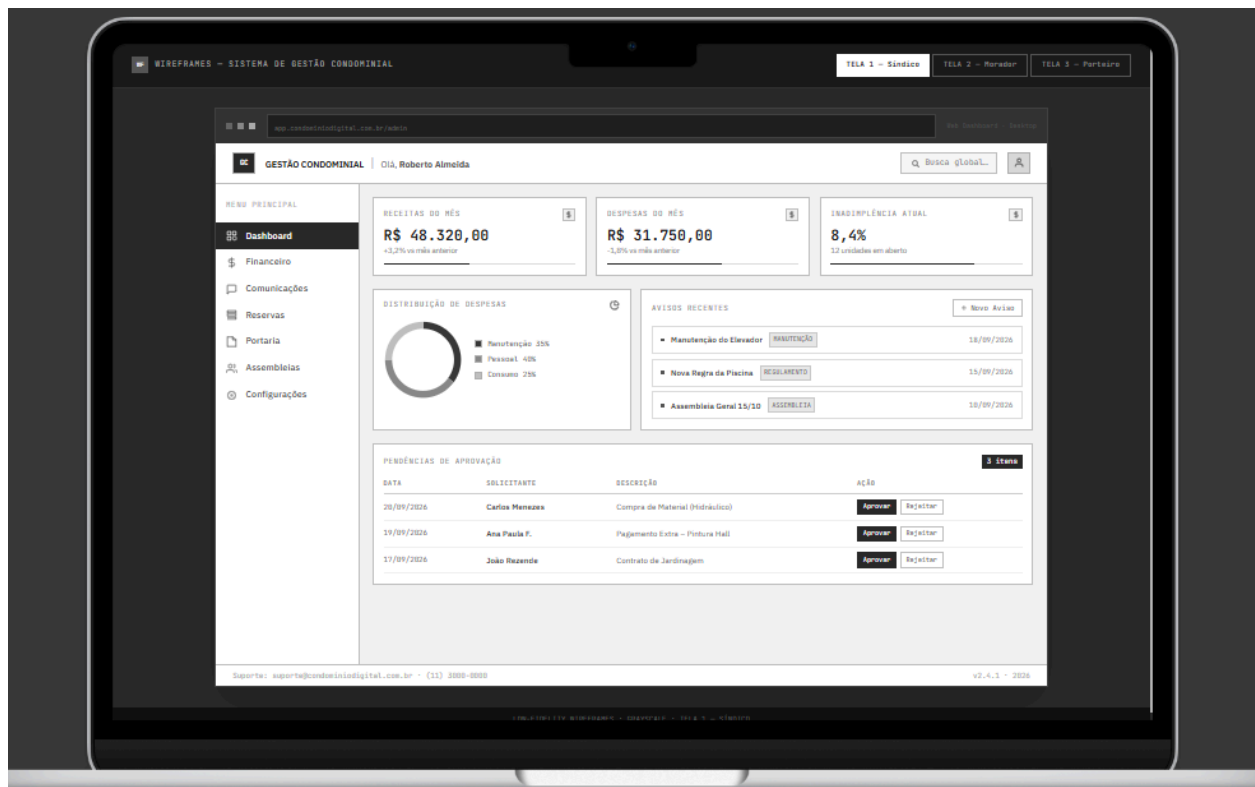
<https://www.figma.com/make/Uc0TzOa6sKBdSimsg0rgEi/Wireframes-para-Gest%C3%A3o-de-Condom%C3%ADnio>



Mapeamento e Detalhamento das Telas

1. Painel do Síndico / Gestão Executiva (Web Dashboard - Desktop)

O **Dashboard do Síndico** centraliza os principais indicadores financeiros e pendências operacionais em uma interface limpa e intuitiva, permitindo decisões rápidas.

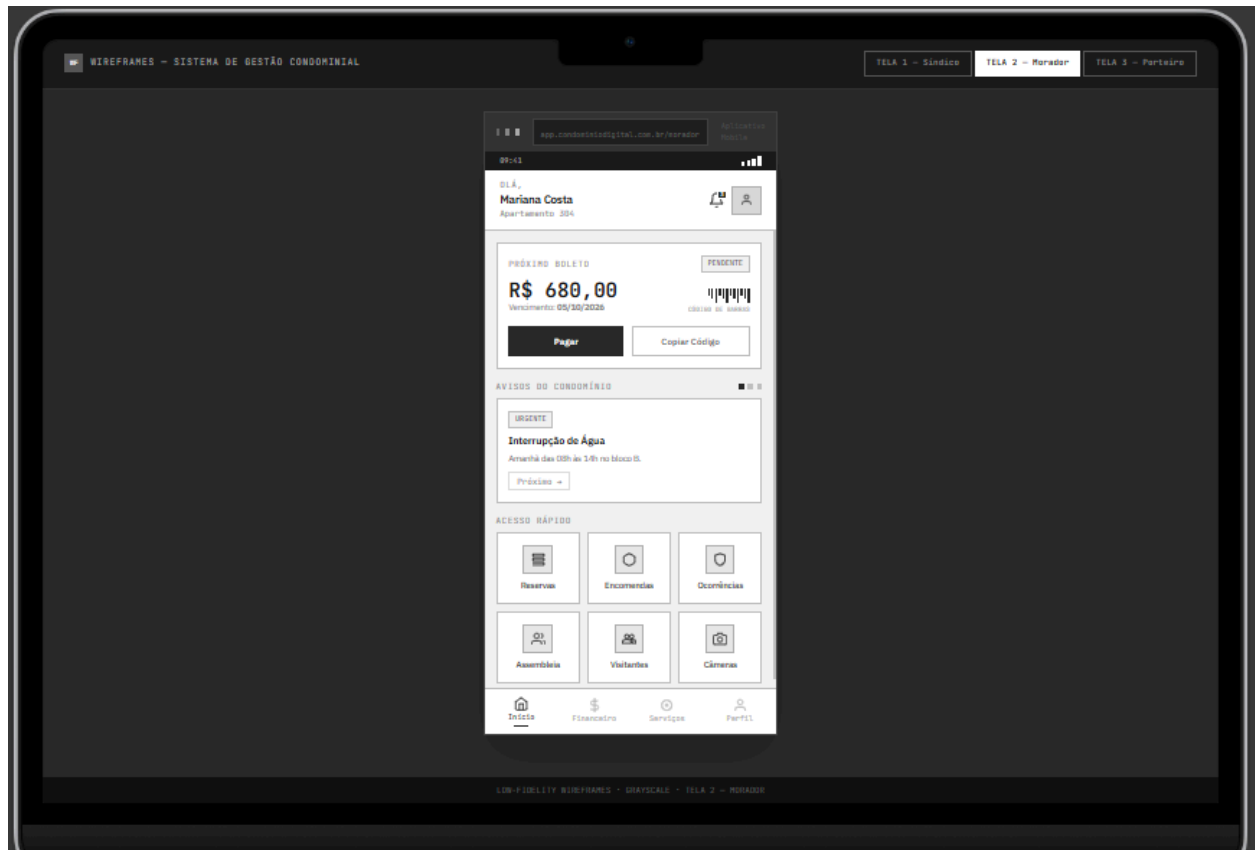


Principais Funcionalidades e Componentes:

- **Métricas Financeiras no Topo (KPIs):**
 - **Receitas do Mês:** Exibição do total arrecadado (ex: *R\$ 48.320,00* com variação percentual vs. mês anterior).
 - **Despesas do Mês:** Acompanhamento dos custos operacionais (*R\$ 31.750,00*).
 - **Taxa de Inadimplência:** Indicador em tempo real (*8,4% - 12 unidades em aberto*).
 - **Distribuição de Despesas:** Gráfico de rosca detalhando os custos por categoria (Manutenção: 35%, Pessoal: 40%, Consumo: 25%).
 - **Mural de Avisos Recentes:** Atalho para criar novos comunicados (*Manutenção do Elevador, Regras da Piscina, Assembleia Geral*).
 - **Pendências de Aprovação:** Tabela de solicitações de compras e serviços (ex: *Materiais, Pintura, Jardinagem*) com ações diretas de **Aprovar** ou **Rejeitar**.
-

2. Aplicativo do Morador (Mobile App - iOS/Android)

O **App do Morador** foca em autonomia e autoatendimento, reunindo os serviços essenciais na palma da mão para evitar chamados desnecessários à administração.



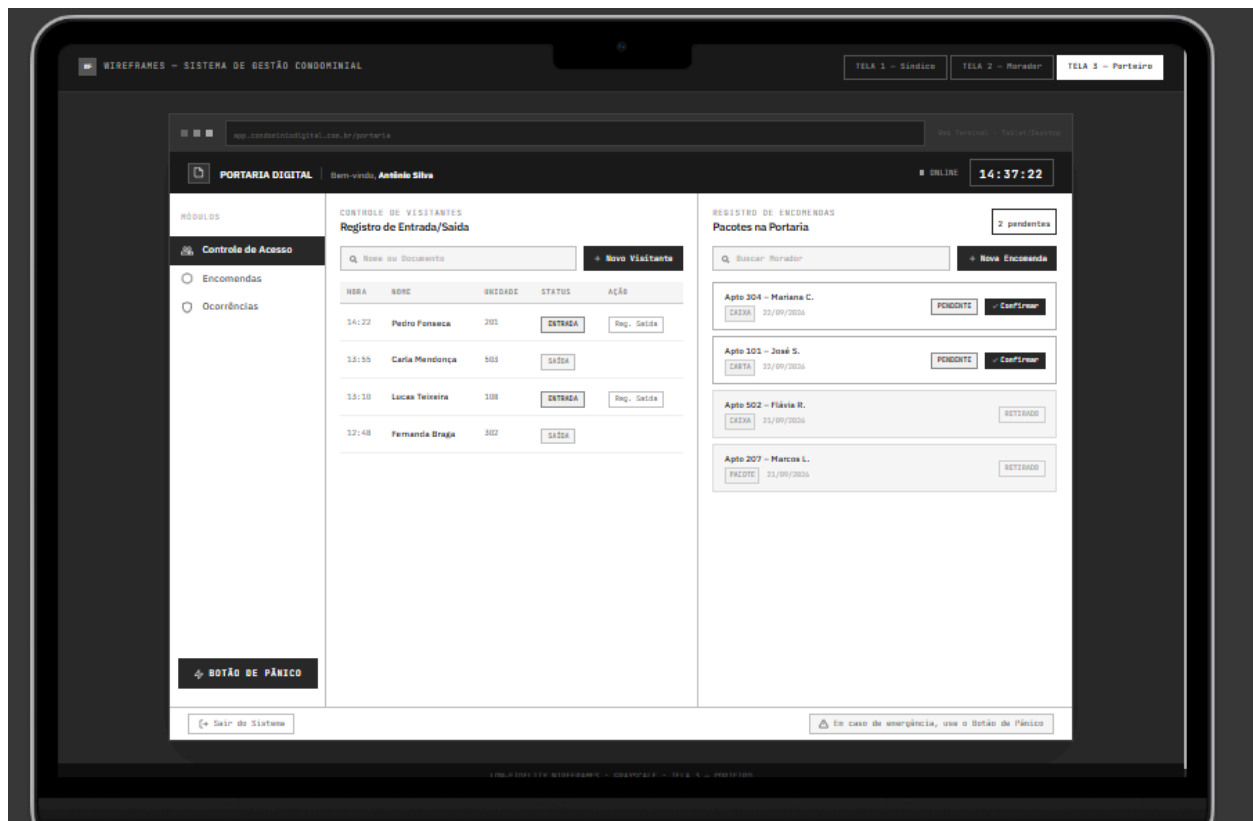
Principais Funcionalidades e Componentes:

- **Card Financeiro em Destaque:** Exibição do próximo boleto a vencer (ex: R\$ 680,00), botão direto para **Pagar via Pix/Boleto** ou **Copiar Código de Barras**.
- **Mural de Avisos Urgentes:** Notificações em tempo real sobre interrupções de serviços no prédio (ex: *Interrupção no fornecimento de água*).
- **Grade de Acesso Rápido (Grid de Serviços):**
 -  **Reservas:** Solicitação de salão de festas e churrasqueiras.
 -  **Encomendas:** Notificação de pacotes recebidos na portaria.
 -  **Ocorrências:** Registro e acompanhamento de chamados.
 -  **Assembleias:** Participação e votações virtuais.
 -  **Visitantes:** Liberação prévia de convidados e prestadores.
 -  **Câmeras:** Visualização ao vivo do circuito interno.

- **Navegação Inferior (Tab Bar):** Navegação fluida entre *Início*, *Financeiro*, *Serviços* e *Perfil*.

3. Terminal da Portaria (Web Terminal - Tablet/Desktop)

O **Terminal de Portaria** foi desenvolvido para ser operado com rapidez e precisão pelos porteiros, focado em segurança e registro instantâneo de movimentações.



🔍 Principais Funcionalidades e Componentes:

- **Controle de Visitantes (Entrada/Saída):**
 - Tabela com horário, nome do visitante, unidade/apartamento e status.
 - Ações rápidas para registrar **Entrada** ou **Registrar Saída** com um clique.
 - Busca global por nome ou documento.
- **Registro Digital de Encomendas (Correspondências):**
 - Lista de pacotes e cartas pendentes de retirada.
 - Notificação direta ao condômino e botão de **Confirmar Entrega** ao morador.

- **Módulo de Emergência:**
 - **Botão de Pânico:** Destaque em caso de ocorrências ou urgências de segurança no condomínio.
-

Tecnologias e Ferramentas

- **Prototipação & UX/UI:** [Figma](#)
- **Modelagem de Negócio:** Business Model Canvas (BMC)
- **Documentação & Versionamento:** Markdown, Git, GitHub

Pilares da Programação Orientada a Objetos (POO)

Resumo teórico produzido para a disciplina de Programação Orientada a Objetos, com base nas referências oficiais da biblioteca UCB.

1. Encapsulamento

❖ Definição:

Consiste em agrupar dados (atributos) e comportamentos (métodos) dentro de uma mesma estrutura (classe), ocultando os detalhes internos de implementação e restringindo o acesso direto ao estado do objeto. O mundo externo interage com o objeto apenas por meio de uma interface pública controlada.

❖ Fundamentação Bibliográfica:

"Encapsulamento é o processo de esconder todos os detalhes de um objeto que não contribuem para as suas características essenciais. Ele envolve a separação da interface de um objeto de sua implementação."

— **SANTOS, Rafael.** *Introdução à Programação Orientada a Objetos Usando Java*, Cap. 1, seção 1.4.

"Encapsulamento [...] é um dos benefícios mais palpáveis de programação orientada a objetos."

— **SANTOS, Rafael.** *Introdução à Programação Orientada a Objetos Usando Java*, Cap. 1, seção 1.4.

❖ Como funciona na prática:

- Os atributos são declarados com modificador de acesso restrito (**private**).
- O acesso e as modificações ocorrem exclusivamente via métodos públicos (*getters*, *setters* ou métodos de regra de negócio).
- Permite validar os dados antes de alterar o estado do objeto (ex.: impedir depósitos negativos ou saldos inválidos).

Exemplo em Java:

```
public class ContaBancaria {  
    private double saldo; // Atributo privado / encapsulado  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    // Método público com validação de regra de negócio  
    public void depositar(double valor) {  
        if (valor > 0) {  
            this.saldo += valor;  
        }  
    }  
}
```

Por que importa: evita que o estado interno do objeto seja corrompido por acessos externos descontrolados, aumentando a segurança e a manutenibilidade do código.

2. Herança

❖ Definição:

Mecanismo da orientação a objetos que permite que uma classe (subclasse ou classe filha) herde atributos (estado) e métodos (comportamento) de outra classe (superclasse ou classe mãe), estabelecendo uma relação hierárquica do tipo "é um" (is-a).

❖ Fundamentação Bibliográfica:

"A herança é um mecanismo que permite a criação de novas classes a partir de classes previamente existentes, herdando seus atributos e métodos, e permitindo a adição de novas funcionalidades ou a modificação do comportamento herdado."

— **SANTOS, Rafael.** *Introdução à Programação Orientada a Objetos Usando Java*, Cap. 7 (Herança).

❖ Como funciona na prática:

- Em Java, a herança é declarada utilizando a palavra-chave **extends**.
- A subclasse adquire os atributos e métodos não privados da superclasse, podendo reaproveitá-los ou adicionar novas características específicas.
- A palavra-chave **super** é utilizada para acessar construtores ou métodos da classe mãe.
- Promove o reuso de código e evita a duplicidade no sistema.

Exemplo em Java:

```
// Superclasse (Classe Mãe)
public class Funcionario {
    private String nome;
    protected double salarioBase;

    public Funcionario(String nome, double salarioBase) {
        this.nome = nome;
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return this.salarioBase;
    }
}

// Subclasse (Classe Filha) - herda estado e comportamento e adiciona novos membros
public class Zelador extends Funcionario {
    private String turno;

    public Zelador(String nome, double salarioBase, String turno) {
        super(nome, salarioBase); // Invoca o construtor da superclasse
        this.turno = turno;
    }

    public String getTurno() {
        return turno;
    }
}
```

Por que importa: promove reuso de código e organização hierárquica entre entidades relacionadas do domínio do problema.

3. Polimorfismo

❖ Definição:

Capacidade de um mesmo método produzir comportamentos diferentes dependendo da instância do objeto que o executa em tempo de execução. Derivado do grego ("muitas formas"), permite tratar objetos de subclasses distintas de maneira genérica e uniforme por meio de uma referência comum da superclasse ou interface.

❖ Fundamentação Bibliográfica:

"Polimorfismo é a capacidade de um mesmo método ser executado de maneiras diferentes dependendo da classe do objeto ao qual a mensagem é enviada em tempo de execução."

— **SANTOS, Rafael.** *Introdução à Programação Orientada a Objetos Usando Java*, Cap. 8.

❖ Como funciona na prática:

- É implementado principalmente por meio da **sobrescrita de métodos** (**@Override**), na qual a subclasse redefine a implementação de um método herdado da superclasse.
- Permite o uso de **ligação dinâmica** (*dynamic binding*): a JVM decide qual versão do método executar apenas no momento da execução, com base no objeto real instanciado na memória, e não no tipo da variável declarada.
- *Nota explicativa:* Embora a sobrecarga de métodos (*overload*) seja por vezes categorizada como polimorfismo estático, o pilar fundamental da POO assenta no polimorfismo dinâmico via sobrescrita.

Exemplo em Java:

```
// Superclasse
public class Funcionario {
    protected double salarioBase;

    public Funcionario(double salarioBase) {
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return this.salarioBase;
    }
}

// Subclasse redefine/sobrescreve o comportamento
public class Zelador extends Funcionario {
    public Zelador(double salarioBase) {
        super(salarioBase);
    }

    @Override
    public double calcularSalario() {
        return super.calcularSalario() * 1.10; // Adicional de 10%
    }
}

// Demonstração do Polimorfismo na prática
public class TestePolimorfismo {
    public static void main(String[] args) {
        // Referência do tipo da superclasse recebendo uma instância da subclasse
        Funcionario f = new Zelador(3000.0);

        // O método executado será o do Zelador (com adicional), decidido em tempo de execução
        System.out.println(f.calcularSalario()); // Saída: 3300.0
    }
}
```

Por que importa: permite tratar objetos de tipos diferentes de forma uniforme (via uma superclasse ou interface comum), tornando o código mais flexível e extensível.

❖ 4. Abstração

❖ Definição:

Processo mental de isolar as características e comportamentos essenciais de uma entidade do mundo real relevantes para o contexto do sistema, ignorando detalhes irrelevantes ou particularidades de implementação. Foca-se no "o que" o objeto faz, em vez de "como" ele faz.

❖ Fundamentação Bibliográfica:

"Abstração é a capacidade de representar as características essenciais de um objeto do mundo real no contexto da aplicação, ignorando os detalhes acidentais ou irrelevantes para o problema em questão."

— **SANTOS, Rafael.** *Introdução à Programação Orientada a Objetos Usando Java*, Cap. 1.

❖ Como se implementa na prática:

- **Classes Abstratas:** Estruturas base que não podem ser instanciadas diretamente (**new**) e que servem como modelos para subclasses, podendo conter métodos abstratos (sem corpo) e métodos concretos.
- **Interfaces:** Contratos puros de comportamento que definem o conjunto de métodos que uma classe concreta obrigatoriamente deve implementar.

Exemplo em Java:

```
// Classe abstrata - define o modelo genérico (não pode ser instanciada diretamente)
public abstract class FormaGeometrica {
    // Método abstrato: define O QUE deve ser feito, sem implementar COMO
    public abstract double calcularArea();
}

// Classe concreta - fornece a implementação específica do conceito abstrato
public class Retangulo extends FormaGeometrica {
    private double largura;
    private double altura;

    public Retangulo(double largura, double altura) {
        this.largura = largura;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return this.largura * this.altura;
    }
}
```

Por que importa: simplifica a complexidade do sistema, permitindo pensar em termos de conceitos gerais antes de se preocupar com os detalhes de implementação.



Referências

- SANTOS, Rafael. *Introdução à Programação Orientada a Objetos Usando Java*. Rio de Janeiro: Elsevier/Campus.
 - MACHADO, Fabrício. *Paradigmas de Programação*.
-